

Assignment 7

Course code **MBAI 5310G AI Programming**

Instructor **Professor Zahra Atf**

Prepared by **Nusrat Jahan**

Student ID **101050785**

Question 1. What does image classification mean? Explain with one example.

Image classification means that I teach a computer to sort a picture under one correct label. The computer studies the colors, the edges, and the overall shape inside the image. Then it picks a single name from a fixed list of options. I have started to see it as a machine with only one job. It answers a single question about the whole picture, which is simply, what is this. The model does not list every object in the scene. It only commits to the one label that fits the full image best.

Here is an example that I find more interesting than the usual ones. Imagine a camera fixed at the entry gate of a parking lot. Each time a vehicle stops in front of it, the camera studies the shape. It then decides whether the vehicle is a motorbike, a private car, or a bus. That label lets the system charge the matching fee. The model has already learned from thousands of past vehicle photos, so now it labels a fresh one in under a second. That single choice is image classification doing its job in real life.

Question 2. Why is using a Dense Network not always a suitable choice for images?

A Dense Network joins every input value to every neuron in the next layer. That design turns heavy the moment I hand it an image. A photo carries thousands of pixels, so the connections multiply at an alarming speed. The model then demands a large amount of memory and training time. I have seen even a small color image push the weight count into the millions, which strikes me as a poor use of resources. The network burns huge effort before it ever learns anything useful.

There is a second weakness that bothers me more. To feed the picture into a Dense Network, I first flatten it into one long line of numbers. That step shreds the image the way a paper shredder cuts a page into thin strips. The simple fact that two pixels were sitting right beside each other now vanishes, even though that closeness is where real shapes live. So the network struggles to read edges, curves, and textures from the scrambled strip. I have learned that convolutional networks were born to solve exactly this problem. They respect the original layout, and they study nearby pixels together.

Question 3. In a CNN, what is convolution, and what does a filter or kernel do?

In a CNN, convolution is a simple action. I take a small window and glide it across the image. At every stop I do a quick round of multiply and add. People call this little window the filter, and sometimes the kernel. The filter is just a tiny grid of numbers. As it travels over the picture, it multiplies its numbers against the pixels below. Then it folds them into one output value. I have followed this motion closely. It reminds me of a small torch that sweeps across a dark wall.

What that torch is hunting for is one specific pattern. One filter may light up at a sharp vertical line, while a different filter may wake up at a soft curve. When the patch beneath the filter matches the numbers stored inside it, the result jumps high. That high value announces that the pattern lives at that exact spot. A real CNN runs dozens of such filters together, so by now it has learned to spot many separate clues. That is how a wall of raw pixels slowly turns into meaning.

Question 4. What is a feature map in a convolutional neural network?

A feature map is the picture of results that one filter leaves behind. I get it once the filter has swept over the whole image. Every cell in this map stores how loudly the filter answered at that spot. So the map behaves like a glowing trail that marks where a chosen pattern shows up. Since each filter builds its own map, one layer quietly produces a tall stack of them. I have come to read a feature map like a thermal camera. The bright spots reveal exactly where the heat, or here the pattern, is hiding.

These maps feed the layers that come after them. They also carry surprisingly rich clues. Near the start of the network, the maps only react to plain things like edges and small shifts in color. Deeper in, the maps have already stitched those plain pieces into fuller shapes, such as a wheel, an eye, or a printed letter. I enjoy watching this climb, because the model builds its understanding floor by floor. The feature maps are the honest receipts of every step it has taken.

Question 5. Why is weight sharing used in CNNs? Explain.

Weight sharing means that I reuse one single filter across the entire image. That one filter keeps the same fixed set of numbers, instead of giving each location its own private weights. I have found that this small decision saves an enormous number of parameters. A leaner parameter count keeps the model light on its feet. So it trains quicker and asks for far less memory. It feels almost thrifty to me. One well trained filter performs a duty that would otherwise swallow thousands of separate weights.

The benefit I admire the most is a different one. The same filter patrols the whole picture. So the network can catch a pattern wherever it appears. I like to picture one sharp building inspector who checks every room himself. He does not hire a fresh inspector for each room. Since it is the very same inspector everywhere, he notices the same crack every time. It does not matter whether the crack sits near the front door or in a far corner. I have learned two names for this idea, and they sound harder than they are. Weight sharing gives translation equivariance in the convolution layer, and it helps the whole network reach translation invariance in its final answer. Equivariance has a simple meaning. When the object moves in the image, its mark in the feature map moves the same way. So the feature travels along with the object. Invariance is just as plain. It means the final answer stays the same. The position of the object does not change it. A bus on the left of the photo and the same bus on the right still get the label bus. This is why weight sharing has made CNNs dependable on messy real images.

Question 6. How does the ReLU activation function work? Write the ReLU output for the values negative 3, 0, 2, and 5.

ReLU is an activation function that I drop onto a neuron output. Its job is to bend the network away from being a plain straight line. Its rule could not be simpler, and I have grown fond of that plainness. When a value is positive, ReLU lets it walk through untouched. When a value is zero or below, ReLU flattens it to zero. I often picture it as a one way valve. That kind of valve lets water flow forward, but it slams shut against anything pushing back.

Let me run the given values through that valve, which are negative 3, then 0, then 2, then 5. Negative 3 falls below zero, so it leaves as 0. The 0 stays as 0. The 2 is positive, so it passes as 2. The 5 is also positive,

so it passes as 5. So the finished ReLU output reads 0, 0, 2, and 5. I have worked through each number on paper, and the same simple rule holds for all of them.

Question 7. What is max pooling, and why is it used in CNNs?

Max pooling is a step that shrinks a feature map. I keep only the strongest value inside each little block. I usually run a two by two window over the map. Out of those four numbers, I save the single biggest one and throw the rest away. As a result, the map loses height and width, yet its loudest evidence survives. I have leaned on this trick often. It trims the data down while protecting the features that matter most.

I reach for max pooling because it pays off in several ways. It shrinks the workload for the deeper layers, since fewer numbers remain to be crunched. It also hands the model a bit of patience toward tiny movements. A feature that slides over by one pixel still lands inside the same block, so it reports the same strong number. To me it works like listening to a noisy classroom and writing down only the loudest voice in each group. That habit has also helped me hold back overfitting in my own runs.

Question 8. In the Fashion MNIST code, why are images reshaped from 28 by 28 to 28 by 28 by 1?

In the Fashion MNIST set, each image shows up as a flat grid of 28 by 28 gray numbers, so it owns only a height and a width. A convolutional layer, though, insists on a third number that names how many channels the image holds. By reshaping the image into 28 by 28 by 1, I attach that missing channel and announce that this is a single channel gray image. I have written this code myself. The convolution layer flatly refuses to run until that lonely 1 is in place.

The deeper point is that the same code is also meant to welcome color images. A color photo turns up as 28 by 28 by 3, since it stacks a red, a green, and a blue channel. The convolution layer always glances at that final number to learn how many channels it must walk through. Fashion MNIST is gray, so it owns just one channel. That is why the shape lands on 28 by 28 by 1. The reshape never edits a single pixel. It only repairs the format. So the data finally fits the doorway that the CNN was built to expect.

Question 9. What is the difference between training data, validation data, and test data?

I cut my data into three groups, because each one carries a separate duty. The training data is the biggest share. The model studies it directly. It tunes its weights over and over, until the patterns sink in. The validation data waits on the sideline during training. I use it to peek at how the model handles brand new examples. This quiet check has guided me while I adjust choices like the layer count or the learning rate. It does all of this without ever touching the final group.

The test data stays sealed until the very last moment. I reveal it to the model only once, after every round of training and tuning is over. Then it returns an honest verdict on real performance. I like to compare the three groups to cooking a brand new dish. The training is me practicing the recipe again and again at home. The validation is my family tasting it, so I can fix the salt. The test is the night I serve it to guests I have never met. If those guests had tasted it during practice, their praise would tell me nothing real.

Question 10. What is overfitting? If training accuracy is high but validation accuracy is low, what does this indicate?

Overfitting happens when a model hugs the training data too tightly. It soaks up the noise and the meaningless little quirks as well. The model has more or less memorized the answer sheet rather than grasping the lesson underneath. So it shines on the data it has already met. Yet it trips the instant I show it something unfamiliar. I have bumped into this many times. It tends to surface when the model is too big. The same thing happens when the training runs for too long.

Sometimes the training accuracy climbs high while the validation accuracy stays low. That very gap is the fingerprint of overfitting. The strong training score says the model knows the seen examples almost by heart. The weak validation score confesses that it crumbles on new ones. In plain words, it has never learned to generalize. I picture a student who memorized the exam paper from last year word for word. He then froze when the teacher changed the questions. To close the gap, I usually gather more data, switch on dropout, add regularization, or stop the training sooner. These moves have rescued me again and again.

Simple Flow Charts:

I have built these charts from my own examples above.

Chart 1. Image classification at a parking gate

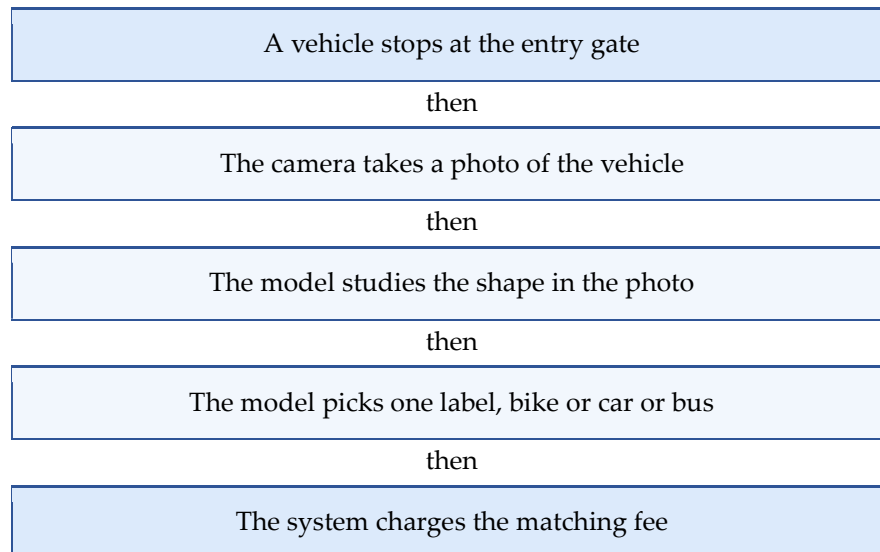


Chart 2. How a CNN reads one image

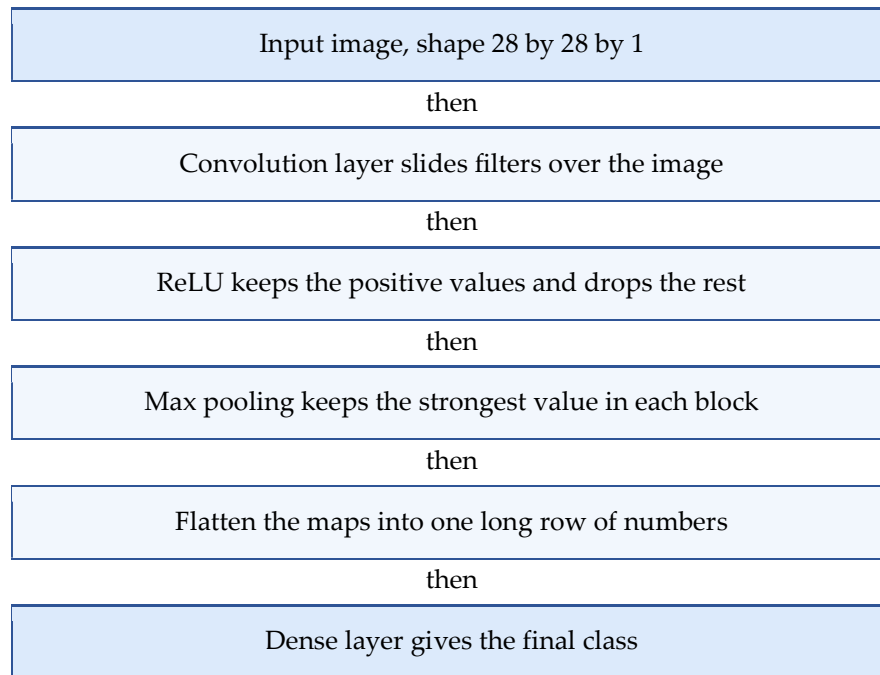


Chart 3. Train, validate, and test the model

